

Do caos à orquestração

Uma aula prática de 45 minutos para sair do “na minha máquina funciona” e chegar a uma infraestrutura multi-serviços declarada em YAML.

45 minutos • 100% na nuvem

Frontend Web (Flask) ↔ Backend (Redis)

Objetivo: entender, declarar, subir, testar e derrubar uma stack multi-serviços.



45 minutos: da dor ao laboratório

O roteiro do README foi mantido como espinha dorsal da apresentação.

0–3

Abertura

“na minha máquina funciona”
+ objetivos

3–10

Fundamentos

containers, VMs,
imagens e camadas

10–15

O problema

6 comandos manuais
e condições frágeis

15–22

A solução

IaC + Compose
+ YAML

22–40

Mão na massa

Killercoda + Flask
+ Redis

40–45

Fech.

Regra de ritmo

Turma júnior → corte “Fundamentos” pela metade e invista no laboratório.

Turma experiente com Docker isolado → pule direto para o problema do gerenciamento imperativo.

Se sobrar tempo: boas práticas + Kubernetes + FAQ

O problema por trás do “na minha máquina funciona”

Dependências, versões e configuração podem variar entre desenvolvimento, laboratório e servidor.

Sem um pacote padronizado...

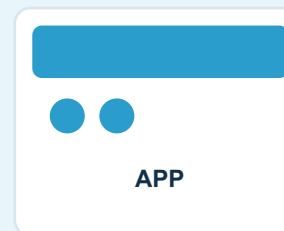
Código “funciona no meu PC”

Bibliotecas versões diferentes

Sistema configuração diferente

Instalação passos manuais

Docker empacota



**Código +
bibliotecas
+ configurações
= unidade
padronizada**

Contêiner

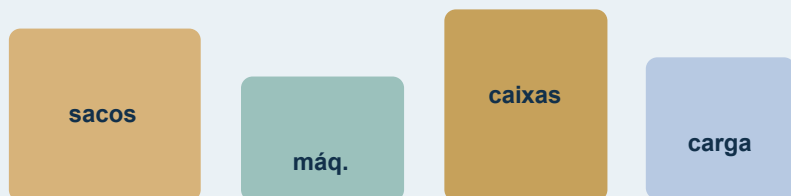
“Caixa” que roda da mesma forma em qualquer ambiente com Docker.

A ideia central: reduzir diferenças do ambiente e tornar a execução previsível.

A analogia do contêiner de aço

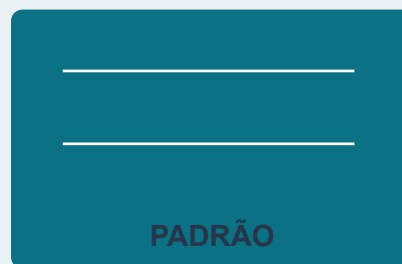
Antes do contêiner padronizado, cargas diferentes exigiam métodos diferentes. O padrão mudou a logística.

Antes



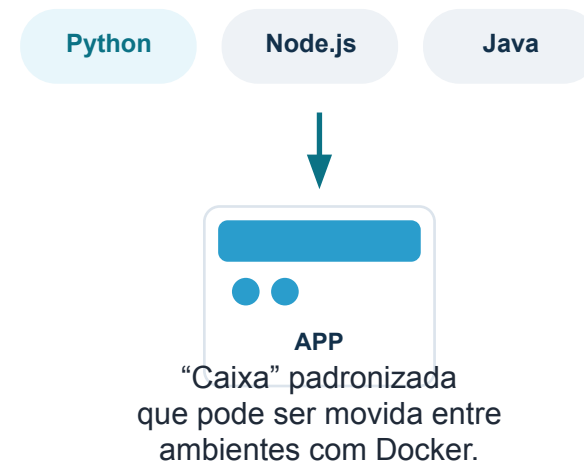
Cada porto lidava com cada carga de um jeito.

Depois



Mesmo tamanho.
Mesmos encaixes.
Qualquer guindaste entende.

Docker faz o mesmo com software



Pergunta para a turma: o que exatamente entra nessa “caixa”?

Arquitetura do Docker: quem fala com quem

O modelo é cliente-servidor, mesmo quando tudo está rodando na mesma máquina.

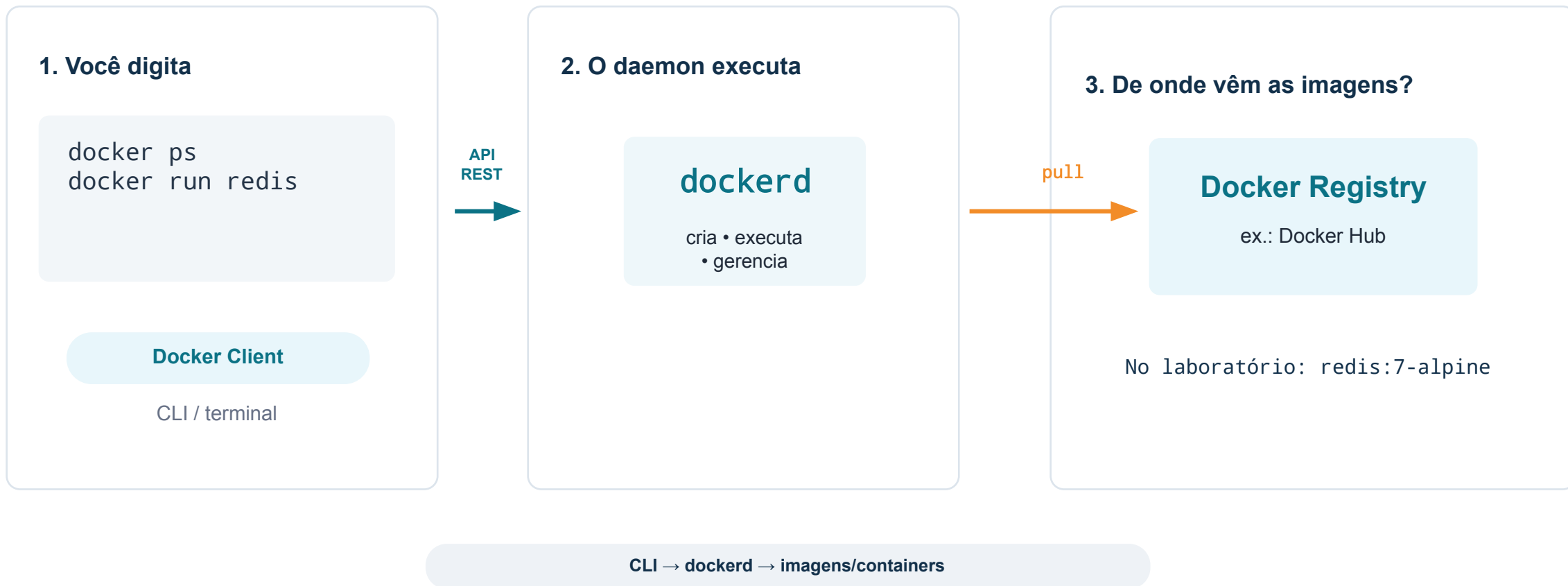


Imagem ≠ Container

Essa distinção evita uma das confusões mais comuns ao começar com Docker.

IMAGEM

“Planta” somente-leitura

```
COPY app.py
```

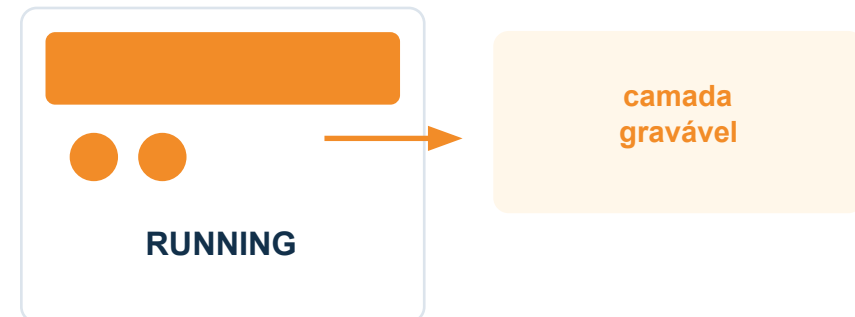
```
RUN pip install...
```

```
FROM python:3.10-alpine
```

Camadas empilhadas • imutável

CONTÊINER

Instância em execução



Efêmero: apagar o container apaga sua camada gravável.

Persistência de dados → volumes

Containers vs. Máquinas Virtuais

A diferença é estrutural: containers compartilham o kernel do host; VMs executam um Guest OS completo.

MÁQUINA VIRTUAL

Hypervisor

Guest OS completo

App + Binários + Libs

Tamanho típico: GB

Boot: minutos

Poucas dezenas / host

CONTÊINER

App + Binários + Libs

Namespaces + cgroups

Kernel do host compartilhado

Tamanho típico: MB

Boot: ms → s

Centenas / host

Não é “uma VM mais rápida”: a arquitetura é diferente.

Por baixo do capô: namespaces + cgroups

Dois mecanismos do kernel Linux explicam boa parte do isolamento e do controle de recursos.

1

Namespaces

Isolam o que cada processo “enxerga”.

rede

processos (PID)

sistema de arquivos

hostname

Cada container tem sua própria visão desses recursos.

2

cgroups

Limitam quanto de recurso um container pode consumir.

CPU

50%

Memória

512 MB

I/O

limites

Evita que um container “faminto” derrube a máquina inteira.

Ponto pedagógico: containers são processos isolados, não máquinas.

Gerenciar a stack só com docker run

A sequência manual funciona, mas depende de flags e etapas que precisam ser lembradas na ordem certa.

```
# 1. criar rede
docker network create frontend-net

# 2. criar volume
docker volume create redis-data

# 3. subir Redis
docker run -d --name redis --network frontend-net \
  -v redis-data:/data redis:7-alpine
```



Flags manuais

--network, -v, -e...



Ordem frágil

sleep 5 ≠ “serviço pronto”



Pouca memória do sistema

a sequência exata não fica em um único lugar

Onde o modelo imperativo começa a falhar

O problema não é o Docker CLI: é transformar estado, dependências e configuração em memória operacional do time.

```
# 4. esperar manualmente
sleep 5

# 5. construir web
docker build -t laboratorio-web .

# 6. subir web
docker run -d --name web --network frontend-net \
  -p 5000:5000 -e REDIS_HOST=redis laboratorio-web
```



Flags manuais

--network, -v, -e...



Ordem frágil

sleep 5 ≠ “serviço pronto”



Pouca memória do sistema

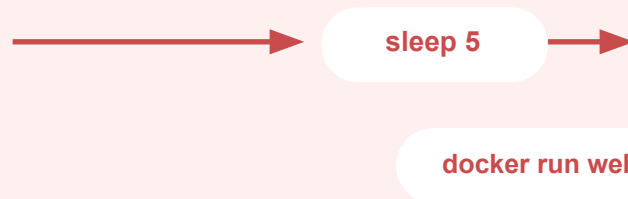
a sequência exata não fica em um único lugar

O “sleep 5” não é uma garantia

Uma máquina sobrecarregada pode precisar de mais tempo; um banco “iniciado” pode ainda não aceitar conexões.

Problema A • ordem de inicialização

`docker run redis`



Cinco segundos são uma aposta, não um mecanismo de prontidão.

Problema B • retry no código

```
while True:
    try:
        return cache.incr('hits')
    except ConnectionError:
        retries -= 1
        time.sleep(0.5)
```

O retry ajuda a aplicação a sobreviver à corrida — mas não substitui uma orquestração correta.

“Retry” é remendo de resiliência; `healthcheck` + `depends_on` atacam a orquestração.

Infraestrutura como Código: declare o estado desejado

Em vez de dizer passo a passo “faça isto, depois aquilo”, descrevemos “quero estes serviços, com estas relações”.

Imperativo

1. criar rede
2. criar volume
3. subir redis
4. esperar
5. buildar web
6. subir web

Você controla a sequência.

Declarativo + IaC

```
services:
  web:
    build: .
  redis:
    image: redis:7-alpine

networks: frontend-net
volumes: redis-data
```

O Compose reconcilia o estado desejado com o estado atual.

Idempotência

rodar de novo → mesmo estado

Versionamento

Git + histórico

Reprodutibilidade

mesma definição

Documentação viva

infra como código

Compose V1, Compose V2 e o campo version

O laboratório usa a forma com hífen por compatibilidade; em instalações novas, o comando sem hífen é o padrão atual.

No laboratório

docker-compose

Compose V1

ferramenta independente,
EOL desde jul/2022

Motivo no guia: ampla compatibilidade com o Killercoda.

Em ambientes novos

docker compose

Compose V2

integrado à CLI do Docker
e mantido ativamente

Sintaxe do YAML: essencialmente a mesma.

E o version: “3.8”?

O README o mantém por clareza didática/compatibilidade; a especificação atual trata o campo como obsoleto.

O arquivo é a planta baixa da infraestrutura

No laboratório, a “planta” define dois serviços: web e redis, mais rede, volume e regras de dependência.

services:	top-level: quais componentes existem
web:	serviço construído localmente
build: .	usa o Dockerfile do diretório atual
ports: 5000:5000	host → container
environment:	injeta REDIS_HOST=redis
depends_on:	controla a dependência do Redis
redis:	serviço baseado em imagem pronta
image: redis:7-alpine	vem do registry
healthcheck:	define “saúde” do serviço
volumes: redis-data	persiste dados fora do container
networks: frontend-net	cria a rede interna

Quatro perguntas para ler YAML

O quê?

services

Como?

build / image

Como conversa?

ports / networks

Como sobrevive?

volumes

Depois entram “quando iniciar?” e “está saudável?”

build vs image • ports • environment

Cada campo responde a uma necessidade diferente do sistema.

build: .

“tijolo”

Constrói a imagem localmente a partir do Dockerfile.

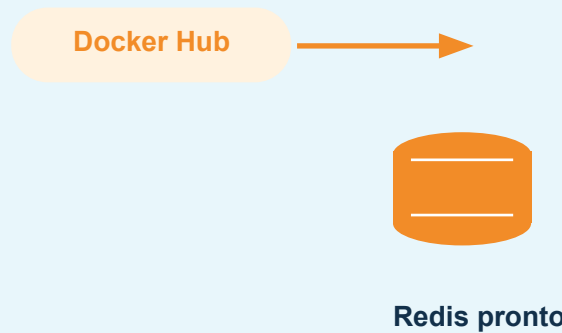
```
web:
  build: .

Dockerfile
  ↓
imagem web
```

image: redis:7-alpine

“pré-fabricado”

Baixa uma imagem pronta do registry; não precisa buildar nada.



ports / environment

5000:5000

túnel: host → container

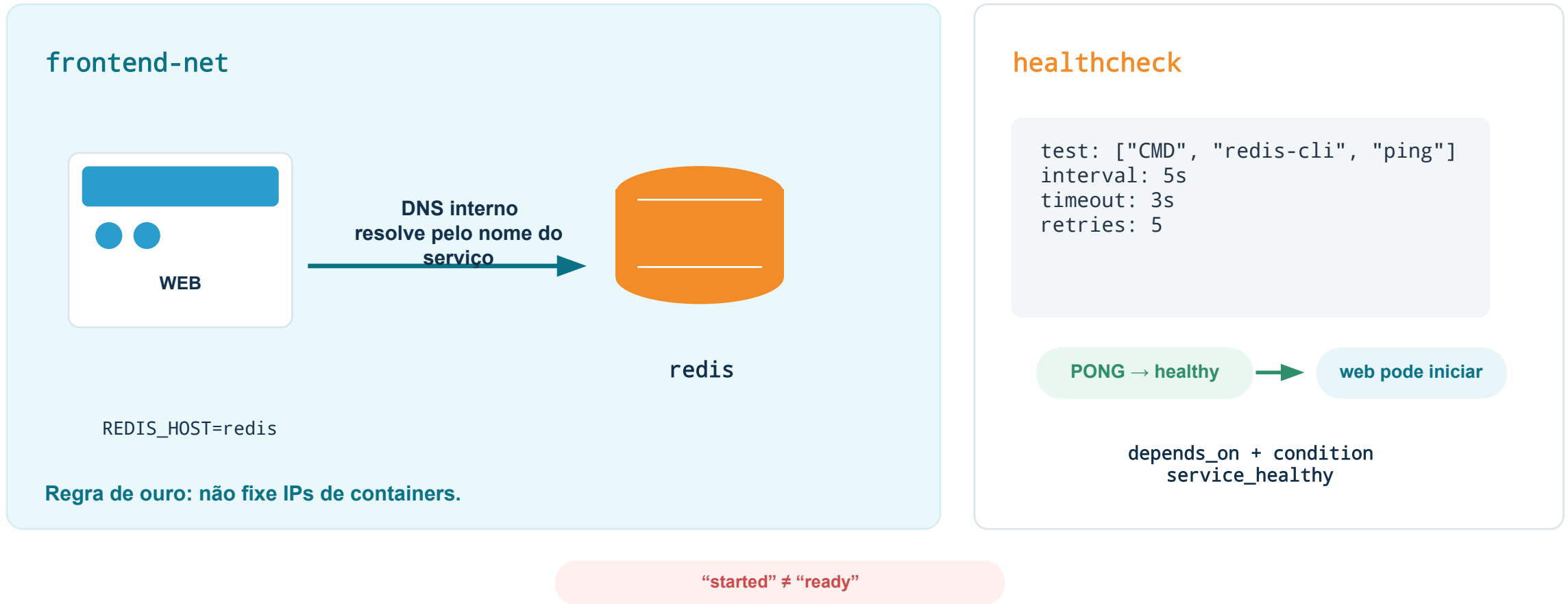
REDIS_HOST=redis

configuração sem mexer no código-fonte

Um serviço pode até combinar build + image.

Rede, healthcheck e depends_on: onde a orquestração começa de verdade

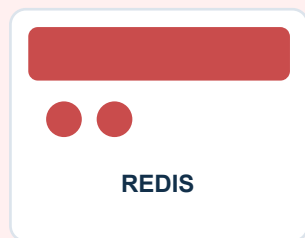
Não basta iniciar containers. O Compose precisa saber como conectá-los e quando o serviço está realmente pronto.



Volumes: o que precisa sobreviver ao container?

Container é efêmero; o estado que importa deve viver fora dele.

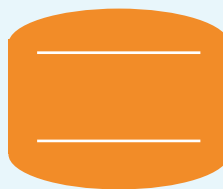
SEM volume



dados dentro da
camada
gravável do container

**Apagou o container?
A camada gravável some.**

VOLUME NOMEADO



redis-data

vive fora do container

BIND MOUNT

```
./app: /code
```

Pasta real do host ↔
Pasta dentro do container

Comum em desenvolvimento
para refletir mudanças de código.

Nosso caso: redis-data:/data

`docker-compose down` → volume sobrevive
`docker-compose down -v` → volume é removido

Killercoda: ambiente sem instalação local

O laboratório acontece em uma VM Ubuntu na nuvem. O Docker já está pronto.

Roteiro de entrada

- 1 Abrir Ubuntu Playground no Killercoda
- 2 Verificar docker-compose --version
- 3 Criar ~/laboratorio-compose
- 4 Seguir o lab passo a passo

⚠ Aviso crítico

**Não pressione F5
nem recarregue a aba.**

O README alerta que isso destrói
a VM instantaneamente.

Sessão: 60 min

Link: Killercoda • Ubuntu Playground

app.py: a aplicação que conversa com Redis

Antes de orquestrar, entenda o comportamento do serviço web e a tentativa de conexão resiliente.

1. app.py

```
redis_host = os.environ.get('REDIS_HOST', 'redis')
cache = redis.Redis(host=redis_host, port=6379)

def get_hit_count():
    retries = 5
    while True:
        try:
            return cache.incr('hits')
        except ConnectionError:
            retries -= 1
            time.sleep(0.5)
```



Resiliência: tenta novamente se o Redis ainda não aceitar conexões.

O que observar

REDIS_HOST

O hostname vem de uma variável de ambiente; o padrão é redis.

retries

Se Redis ainda não aceita conexões, a função tenta novamente.

incr('hits')

Cada acesso incrementa o contador armazenado no Redis.

0.0.0.0:5000

A aplicação escuta em todas as interfaces do container.

Dockerfile + build: empacotando a aplicação

O Dockerfile é a receita da imagem web; o build transforma essa receita em uma imagem utilizável pelo Compose.

2. Dockerfile

```
FROM python:3.10-alpine
WORKDIR /code
RUN pip install flask redis
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

“Receita de bolo” da imagem web.

3. O build

python:3.10-alpine



WEB

imagem pronta para
ser usada pelo Compose

Compose YAML: declarando os serviços

Começamos pelo que será executado: o web é buildado localmente e o Redis usa uma imagem pronta.

```
version: '3.8'
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - REDIS_HOST=redis
    depends_on:
      redis:
        condition: service_healthy
    networks:
      - frontend-net

  redis:
    image: redis:7-alpine
```

Leia isto como estado desejado: quais serviços existem e como o web depende do Redis.

3 conexões-chave

REDE web ↔ redis
frontend-net

PRONTIDÃO healthcheck + depends_on
service_healthy

PERSISTÊNCIA Redis → volume
redis-data:/data

Compose YAML: prontidão, rede e persistência

É aqui que o manifesto deixa explícito quando um serviço está pronto, como os serviços se conectam e onde o estado sobrevive.

```
redis:
  image: redis:7-alpine
  ports:
    - "6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 3s
    retries: 5
  networks:
    - frontend-net
  volumes:
    - redis-data:/data

networks:
  frontend-net:
    driver: bridge

volumes:
  redis-data:
```

Leia como um estado desejado: serviços + relações + recursos.

3 conexões-chave

REDE web ↔ redis
frontend-net

PRONTIDÃO healthcheck + depends_on
service_healthy

PERSISTÊNCIA Redis → volume
redis-data:/data

Um comando. Dois serviços. Uma stack.

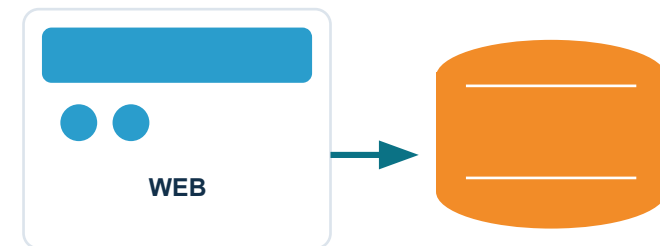
A partir daqui, a turma pode observar o mesmo estado desejado virar containers reais.

```
docker-compose up -d
```

- 1 sobe os serviços em background (detached)

```
docker-compose logs -f
```

- 2 acompanha logs combinados; Ctrl+C sai dos logs sem parar containers



Traffic / Ports

porta 5000 → interface no navegador

```
curl http://localhost:5000
```

Teste a aplicação e provoque uma falha

Primeiro valide a comunicação web → Redis; depois reinicie apenas o banco e observe o estado persistido.

1 • TESTAR

```
curl http://localhost:5000
```

O contador deve aumentar a cada requisição.

Web (Flask) conversa com Redis pela rede interna.

2 • SIMULAR “CAOS”

```
docker-compose restart redis
```

Reinicie apenas o Redis.

Depois rode curl novamente.

Resultado esperado do passo 2: o contador continua de onde parou → redis-data preservou o estado.

Demolição limpa: o que o -v realmente faz

Encerramos o laboratório removendo containers, rede e volume — porque o recurso persistente só some quando pedimos isso explicitamente.

3 • DEMOLIR

```
docker-compose down -v
```

Remove containers, rede e volumes.

No laboratório, -v é intencional: precisamos limpar o ambiente.

Aprendizado central: execução é efêmera; estado persistente fica fora do ciclo de vida do container.

Resultado esperado do passo 2: o contador continua de onde parou → redis-data preservou o estado.

Boas práticas: o laboratório já contém várias delas

Use esta tela como checklist de revisão e não como uma nova aula de infraestrutura.



Fixar tags

redis:7-alpine • python:3.10-alpine



Segredos fora do YAML

.env no .gitignore / secret manager



Expor só o necessário

web público; Redis via rede interna



healthcheck

dependências precisam de sinais de prontidão



.dockerignore + limpeza

evita contexto desnecessário e acúmulo



Validar antes de subir

docker-compose config

“docker compose up” deve ser repetível; não dependa da memória de alguém.

Pergunta de encerramento → o que está descrito no YAML e o que continua sendo responsabilidade do código?

Compose → Swarm → Kubernetes

A mentalidade declarativa que praticamos hoje é a mesma base que aparece em orquestradores de cluster.

Docker Compose

single-host

- 1 host
- Auto-healing: não
- Escala: manual (--scale)
- Dev local • CI/CD • apps pequenas

Docker Swarm

cluster

- múltiplos hosts
- Auto-healing: sim
- Auto-scaling: limitado
- Times já centrados em Docker

Kubernetes

cluster

- múltiplos hosts
- Auto-healing: sim
- HPA / escala avançada
- Produção em escala

Sinais de “hora de migrar”: mais de um servidor, failover automático, multi-tenancy, ou scaling manual frequente.

FAQ: dúvidas que consolidam a aula

Use estas perguntas para verificar se a turma entendeu o papel de Dockerfile, Compose, volumes e escala.

FAQ

Dockerfile substitui Compose?	Não. Dockerfile = construir imagem; Compose = orquestrar serviços.
Por que dados sumiram?	Provavelmente foi usado <code>down -v</code> , que remove volumes.
docker-compose ou docker compose?	Em ambiente novo, prefira <code>docker compose</code> ; o laboratório usa a forma com hífen.
Posso escalar web?	Sim, com <code>--scale</code> ; porta fixa pode causar conflito.
Editar YAML muda na hora?	Não. Rode <code>up -d</code> novamente para reconciliar o estado.

Mensagem final: declare a infraestrutura; deixe o Compose cuidar da coordenação.

Comandos essenciais para a stack

Subir, inspecionar, acompanhar logs, reiniciar, validar e limpar a infraestrutura.

CHEAT SHEET

```
docker-compose up -d
```

```
docker-compose ps
```

```
docker-compose logs -f
```

```
docker-compose restart redis
```

```
docker-compose config
```

```
docker-compose down
```

```
docker-compose down -v
```

```
docker-compose up -d --scale web=N
```

Mensagem final: declare a infraestrutura; deixe o Compose cuidar da coordenação.